

SISTEMA E-COMMERCE MICROSERVICIOS

Documentación Completa de Endpoints — API Reference

Ingeniería de Software II — Entrega #1
Fecha: 3 de abril de 2026

Resumen del Sistema

Microservicio	Framework	Base de Datos	Puerto
API Gateway	Laravel PHP 8.3	MySQL	8000
Usuarios	Django 5.x	PostgreSQL	8001
Productos	Flask	Firebase	8002
Órdenes	Express Node.js	MongoDB	8003
Pagos	Django 5.x	PostgreSQL	8004

1. API Gateway — Autenticación

Base URL: `http://localhost:8000/api`

El Gateway es el único punto de entrada al sistema. Gestiona autenticación JWT y hace proxy hacia los microservicios internos.

Resumen de Endpoints

POST	<code>/api/register</code>	Registrar nuevo usuario	Público
POST	<code>/api/login</code>	Iniciar sesión — retorna JWT	Público
POST	<code>/api/logout</code>	Cerrar sesión — invalida el token	JWT
GET	<code>/api/me</code>	Datos del usuario autenticado	JWT
POST	<code>/api/forgot-password</code>	Enviar enlace de recuperación	Público
POST	<code>/api/reset-password</code>	Restablecer contraseña con token	Público

POST `/api/register`

Registra un nuevo usuario en la base de datos del Gateway (MySQL). Valida que el email sea único y que las contraseñas coincidan. No requiere autenticación.

Parámetros del body (JSON)

Campo	Tipo	Requerido	Descripción
<code>name</code>	<code>string</code>	✓ Sí	Nombre completo del usuario. Máximo 255 caracteres.
<code>email</code>	<code>string</code>	✓ Sí	Correo electrónico único. Debe tener formato válido (<code>user@domain.com</code>).
<code>password</code>	<code>string</code>	✓ Sí	Contraseña. Mínimo 8 caracteres.
<code>password_confirmation</code>	<code>string</code>	✓ Sí	Confirmación de contraseña. Debe coincidir exactamente con <code>password</code> .

Ejemplo de petición

```
POST /api/register
Content-Type: application/json

{
  "name": "Juan Pérez",
  "email": "juan@example.com",
  "password": "password123"
}
```

Respuestas

Status	Descripción y ejemplo
201	Registro exitoso. Retorna mensaje de confirmación y datos básicos del usuario (id, name, email).
422	Validación fallida. Puede ser: email ya registrado, contraseñas no coinciden, password muy corta, campos faltantes.

Ejemplo de respuesta exitosa (201)

```
{
  "message": "Usuario registrado exitosamente",
  "user": {
    "id": 1,
    "name": "Juan Pérez",
    "email": "juan@example.com"
  }
}
```

Ejemplo de error (422)

```
{
  "message": "The email has already been taken.",
  "errors": {
    "email": ["The email has already been taken."]
  }
}
```

POST /api/login

Autentica al usuario y retorna un token JWT (JSON Web Token) firmado con HS256. El token tiene una vigencia de 60 minutos y debe incluirse en todas las peticiones protegidas.

Parámetros del body (JSON)

Campo	Tipo	Requerido	Descripción
email	string	✓ Sí	Correo electrónico registrado en el sistema.
password	string	✓ Sí	Contraseña del usuario.

Ejemplo de petición

```
POST /api/login
Content-Type: application/json
Accept: application/json

{
  "email": "juan@example.com",
  "password": "password123"
}
```

Respuestas

Status	Descripción y ejemplo
200	Login exitoso. Retorna access_token (JWT), token_type "bearer" y expires_in en segundos.
401	Credenciales inválidas. Email o contraseña incorrectos.
422	Campos faltantes o formato inválido.

Ejemplo de respuesta exitosa (200)

```
{
  "access_token": "eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9...",
  "token_type": "bearer",
  "expires_in": 3600
}
```

IMPORTANTE: Guarda el access_token. Debes enviarlo en todas las peticiones protegidas así:

```
Authorization: Bearer eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9...
```

POST /api/logout

Invalida el token JWT actual en el servidor. Después de hacer logout, el token no puede usarse para autenticar más peticiones. Requiere el token en el header Authorization.

Ejemplo de petición

```
POST /api/logout
Authorization: Bearer eyJ0eXAiOiJKV1Qi...
Accept: application/json
```

Respuestas

Status	Descripción y ejemplo
200	Sesión cerrada exitosamente.
401	Token no proporcionado, inválido o expirado.

Ejemplo de respuesta (200)

```
{ "message": "Sesión cerrada exitosamente" }
```

GET /api/me

Retorna los datos completos del usuario actualmente autenticado, basándose en el token JWT enviado.

Ejemplo de petición

```
GET /api/me
Authorization: Bearer eyJ0eXAiOiJKV1Qi...
Accept: application/json
```

Respuestas

Status	Descripción y ejemplo
200	Datos del usuario autenticado: id, name, email, email_verified_at, created_at, updated_at.
401	Token no proporcionado, inválido o expirado.

Ejemplo de respuesta (200)

```
{
  "id": 1,
  "name": "Juan Pérez",
  "email": "juan@example.com",
  "email_verified_at": null,
  "created_at": "2026-04-01T20:00:00.000000Z",
  "updated_at": "2026-04-01T20:00:00.000000Z"
}
```

}

POST /api/forgot-password

Envía un enlace de recuperación de contraseña al correo del usuario. Requiere que el servidor de correo (SMTP) esté configurado en el .env del Gateway.

Parámetros del body (JSON)

Campo	Tipo	Requerido	Descripción
email	string	✓ Sí	Correo electrónico del usuario que quiere recuperar su contraseña.

Respuestas

Status	Descripción y ejemplo
200	Enlace enviado correctamente al correo.
400	No se pudo enviar el enlace (correo no encontrado o error SMTP).
422	Email con formato inválido o campo faltante.

POST /api/reset-password

Restablece la contraseña usando el token recibido por correo electrónico. El token tiene un tiempo de expiración definido en la configuración de Laravel.

Parámetros del body (JSON)

Campo	Tipo	Requerido	Descripción
token	string	✓ Sí	Token recibido en el correo de recuperación.
email	string	✓ Sí	Correo electrónico del usuario.
password	string	✓ Sí	Nueva contraseña. Mínimo 6 caracteres.
password_confirmation	string	✓ Sí	Confirmación de la nueva contraseña.

Respuestas

Status	Descripción y ejemplo
200	Contraseña actualizada correctamente.
400	Token inválido, expirado o email incorrecto.
422	Validación fallida en los campos enviados.

2. Microservicio Usuarios

Framework: Django 5.x | Base de datos: PostgreSQL | Puerto interno: 8001

Acceso vía Gateway: <http://localhost:8000/api/usuarios>

Acceso directo (requiere X-Internal-Key): <http://localhost:8001/usuarios>

Gestiona el ciclo de vida de los usuarios del e-commerce. Implementa soft delete (desactivación) en lugar de eliminar físicamente los registros. Todas las peticiones directas al puerto 8001 deben incluir el header X-Internal-Key.

Resumen de Endpoints

GET	/api/usuarios	Listar todos los usuarios activos	JWT
POST	/api/usuarios	Crear nuevo usuario	JWT
GET	/api/usuarios/{id}	Obtener usuario por ID	JWT
PUT	/api/usuarios/{id}	Actualizar datos del usuario	JWT
DELETE	/api/usuarios/{id}	Desactivar usuario (soft delete)	JWT

GET /api/usuarios

Retorna todos los usuarios con estado activo=true. Los usuarios desactivados con DELETE no aparecen en este listado.

Parámetros de query (opcionales)

Campo	Tipo	Requerido	Descripción
(ninguno)	-	☐ No	No acepta filtros por query params actualmente.

Respuestas

Status	Descripción y ejemplo
200	Lista de usuarios activos. Puede ser un arreglo vacío si no hay usuarios.
401	Token JWT no proporcionado o inválido.
403	Acceso directo sin X-Internal-Key (solo aplica al puerto 8001).

Ejemplo de respuesta (200)

```
{
  "usuarios": [
    {
      "id": 1,
      "nombre": "Ana García",
      "email": "ana@example.com",
      "telefono": "3001234567",
      "direccion": "Calle 10 # 20-30, Manizales",
      "activo": true,
      "creado_en": "2026-04-01T20:00:00Z"
    }
  ]
}
```

POST /api/usuarios

Crea un nuevo usuario en la base de datos PostgreSQL. Valida que el email no esté duplicado y que los campos requeridos estén presentes.

Parámetros del body (JSON)

Campo	Tipo	Requerido	Descripción
nombre	string	✓ Sí	Nombre completo del usuario.
email	string	✓ Sí	Correo electrónico. Debe ser único en el sistema.
telefono	string	○ No	Número de teléfono de contacto.
direccion	string	○ No	Dirección de envío del usuario.

Ejemplo de petición

```
POST /api/usuarios
Authorization: Bearer eyJ0eXAiOiJKV1Qi...
Content-Type: application/json

{
  "nombre": "Ana García",
```

```
{
  "email": "ana@example.com",
  "telefono": "3001234567",
  "direccion": "Calle 10 # 20-30, Manizales"
}
```

Respuestas

Status	Descripción y ejemplo
201	Usuario creado exitosamente. Retorna el objeto completo del usuario con su id asignado.
422	Error de validación: nombre o email faltante, o email duplicado.
400	JSON malformado en el body.
401	Token JWT inválido o faltante.

Ejemplo de respuesta (201)

```
{
  "id": 1,
  "nombre": "Ana García",
  "email": "ana@example.com",
  "telefono": "3001234567",
  "direccion": "Calle 10 # 20-30, Manizales",
  "activo": true,
  "creado_en": "2026-04-01T20:00:00Z"
}
```

Ejemplo de error (422 — email duplicado)

```
{ "error": "El email ya existe" }
```

GET /api/usuarios/{id}

Obtiene un usuario específico por su ID de PostgreSQL. Retorna 404 si el usuario no existe.

Parámetros de ruta

Campo	Tipo	Requerido	Descripción
id	integer	✓ Sí	ID numérico del usuario en la base de datos PostgreSQL.

Respuestas

Status	Descripción y ejemplo
200	Datos completos del usuario.
404	{ "error": "Usuario no encontrado" }
401	Token JWT inválido o faltante.

PUT /api/usuarios/{id}

Actualiza parcialmente los datos de un usuario. Solo se pueden modificar nombre, telefono y direccion. El email no es modificable para mantener integridad.

Parámetros del body (todos opcionales)

Campo	Tipo	Requerido	Descripción
nombre	string	o No	Nuevo nombre del usuario.
telefono	string	o No	Nuevo número de teléfono.
direccion	string	o No	Nueva dirección de envío.

Respuestas

Status	Descripción y ejemplo
200	Usuario actualizado. Retorna el objeto completo con los nuevos datos.
404	{ "error": "Usuario no encontrado" }
400	JSON malformado.
401	Token JWT inválido o faltante.

DELETE /api/usuarios/{id}

Desactiva un usuario cambiando su campo activo a false (soft delete). El usuario no se elimina físicamente de la base de datos y no aparecerá en el listado de GET /usuarios.

Respuestas

Status	Descripción y ejemplo
200	{ "message": "Usuario desactivado" }
404	{ "error": "Usuario no encontrado" }
401	Token JWT inválido o faltante.

3. Microservicio Productos

Framework: Flask | Base de datos: Firebase Realtime Database | Puerto interno: 8002

Acceso vía Gateway: <http://localhost:8000/api/productos>

Gestiona el catálogo de productos del e-commerce usando Firebase como base de datos en tiempo real. Los IDs de productos son generados por Firebase (strings como -NxK2abc123) en lugar de enteros secuenciales.

Resumen de Endpoints

GET	/api/productos	Listar todos los productos	JWT
POST	/api/productos	Crear nuevo producto	JWT
GET	/api/productos/{id}	Obtener producto por ID de Firebase	JWT
PUT	/api/productos/{id}	Actualizar campos del producto	JWT
DELETE	/api/productos/{id}	Eliminar producto de Firebase	JWT

GET /api/productos

Retorna todos los productos almacenados en Firebase Realtime Database. Si no hay productos, retorna un arreglo vacío.

Respuestas

Status	Descripción y ejemplo
200	Lista de productos. Cada producto incluye su id de Firebase y todos sus campos.
401	Token JWT inválido.
403	Acceso directo sin X-Internal-Key.

Ejemplo de respuesta (200)

```
{
  "productos": [
    {
      "id": "-NxK2abc123",
      "nombre": "Laptop Dell XPS 15",
      "descripcion": "Procesador Intel Core i7, 16GB RAM, 512GB SSD",
      "precio": 1200.00,
      "stock": 50,
      "categoria": "Tecnología"
    }
  ]
}
```

POST /api/productos

Crea un nuevo producto en Firebase. Valida que el nombre y precio estén presentes, y que el precio no sea negativo.

Parámetros del body (JSON)

Campo	Tipo	Requerido	Descripción
nombre	string	✓ Sí	Nombre del producto. No puede estar vacío.
precio	number	✓ Sí	Precio unitario. Debe ser mayor o igual a 0.
stock	integer	○ No	Cantidad disponible en inventario. Default: 0.
categoria	string	○ No	Categoría del producto (ej: Tecnología, Ropa, Hogar).
descripcion	string	○ No	Descripción detallada del producto.

Respuestas

Status	Descripción y ejemplo
201	Producto creado. Retorna el objeto con el id de Firebase asignado.
422	Validación fallida: nombre faltante, precio faltante o precio negativo.
401	Token JWT inválido.

Ejemplo de respuesta (201)

```
{
  "id": "-NxK2abc123",
```

```
"nombre": "Laptop Dell XPS 15",
"precio": 1200.00,
"stock": 50,
"categoria": "Tecnología",
"descripcion": "Procesador Intel Core i7"
}
```

Ejemplo de error (422)

```
{
  "errors": {
    "nombre": "El nombre es requerido",
    "precio": "El precio es requerido"
  }
}
```

GET /api/productos/{id}

Obtiene un producto específico por su ID de Firebase. El ID es un string generado por Firebase (ej: -NxK2abc123).

Parámetros de ruta

Campo	Tipo	Requerido	Descripción
id	string	✓ Sí	ID de Firebase del producto (ej: -NxK2abc123).

Respuestas

Status	Descripción y ejemplo
200	Datos completos del producto.
404	{ "error": "Producto no encontrado" }

PUT /api/productos/{id}

Actualiza uno o más campos de un producto existente en Firebase. Solo se envían los campos a modificar.

Parámetros del body (todos opcionales)

Campo	Tipo	Requerido	Descripción
nombre	string	☐ No	Nuevo nombre del producto.
precio	number	☐ No	Nuevo precio del producto.
stock	integer	☐ No	Nueva cantidad en inventario.
categoria	string	☐ No	Nueva categoría.
descripcion	string	☐ No	Nueva descripción.

Respuestas

Status	Descripción y ejemplo
200	Producto actualizado. Retorna el objeto completo con los nuevos valores.
404	{ "error": "Producto no encontrado" }

DELETE /api/productos/{id}

Elimina un producto permanentemente de Firebase Realtime Database. Esta operación no es reversible.

Respuestas

Status	Descripción y ejemplo
200	{ "message": "Producto eliminado" }
404	{ "error": "Producto no encontrado" }

4. Microservicio Órdenes

Framework: Express (Node.js) | Base de datos: MongoDB | Puerto interno: 8003
Acceso vía Gateway: <http://localhost:8000/api/ordenes>

Gestiona las órdenes de compra del e-commerce. Usa MongoDB como base de datos NoSQL, por lo que los IDs son ObjectId de MongoDB (strings hexadecimales de 24 caracteres). El total de la orden se calcula automáticamente multiplicando cantidad × precio_unitario de cada producto.

Resumen de Endpoints

GET	/api/ordenes	Listar órdenes (filtrable por usuario)	JWT
POST	/api/ordenes	Crear nueva orden	JWT
GET	/api/ordenes/{id}	Obtener orden por ObjectId de MongoDB	JWT
PUT	/api/ordenes/{id}	Actualizar estado o dirección de orden	JWT
DELETE	/api/ordenes/{id}	Cancelar orden	JWT

GET /api/ordenes

Lista todas las órdenes ordenadas por fecha de creación descendente. Acepta un query param para filtrar por usuario.

Parámetros de query (opcionales)

Campo	Tipo	Requerido	Descripción
usuario_id	integer	○ No	Filtra las órdenes de un usuario específico. Ej: <code>/api/ordenes?usuario_id=1</code>

Respuestas

Status	Descripción y ejemplo
200	Lista de órdenes. Puede estar vacía si no hay órdenes registradas.
401	Token JWT inválido.

Ejemplo de respuesta (200)

```
{
  "ordenes": [
    {
      "_id": "69cd838222e0b8cdbf1d7998",
      "usuario_id": 1,
      "productos": [
        {
          "producto_id": "prod-001",
          "nombre": "Laptop Dell XPS",
          "cantidad": 2,
          "precio_unitario": 1200,
          "_id": "69cd838222e0b8cdbf1d7999"
        }
      ],
      "total": 2400,
      "estado": "pendiente",
      "direccion_envio": "Calle 10 # 20-30, Manizales",
      "creado_en": "2026-04-01T20:43:46.821Z",
      "actualizado_en": "2026-04-01T20:43:46.821Z",
      "__v": 0
    }
  ]
}
```

POST /api/ordenes

Crea una nueva orden. El campo total se calcula automáticamente sumando (cantidad × precio_unitario) de cada producto en el arreglo. No es necesario enviarlo.

Parámetros del body (JSON)

Campo	Tipo	Requerido	Descripción
usuario_id	integer	✓ Sí	ID del usuario que realiza la orden.
productos	array	✓ Sí	Arreglo de productos. Debe tener al menos un elemento.
producto_id	string	✓ Sí	ID del producto (dentro del arreglo productos).
nombre	string	✓ Sí	Nombre del producto (dentro del arreglo productos).

Campo	Tipo	Requerido	Descripción
<code>cantidad</code>	<code>integer</code>	✓ Sí	Cantidad a ordenar. Mínimo 1.
<code>precio_unitario</code>	<code>number</code>	✓ Sí	Precio por unidad. Mínimo 0.
<code>direccion_envio</code>	<code>string</code>	○ No	Dirección de entrega. Default: cadena vacía.

Ejemplo de petición

```
POST /api/ordenes
Authorization: Bearer eyJ0eXAiOiJKV1Qi...
Content-Type: application/json

{
  "usuario_id": 1,
  "productos": [
    {
      "producto_id": "prod-001",
      "nombre": "Laptop Dell XPS",
      "cantidad": 2,
      "precio_unitario": 1200.00
    }
  ],
  "direccion_envio": "Calle 10 # 20-30, Manizales"
}
```

Respuestas

Status	Descripción y ejemplo
201	Orden creada. Retorna el objeto completo con <code>_id</code> de MongoDB y total calculado automáticamente.
422	Validación fallida: <code>usuario_id</code> faltante o arreglo productos vacío.
401	Token JWT inválido.

Estados válidos de una orden

Campo	Tipo	Requerido	Descripción
<code>pendiente</code>	-	○ No	Estado inicial al crear la orden.
<code>confirmada</code>	-	○ No	Orden confirmada por el sistema.
<code>enviada</code>	-	○ No	Orden despachada al transportista.
<code>entregada</code>	-	○ No	Orden recibida por el cliente.
<code>cancelada</code>	-	○ No	Orden cancelada. Se asigna al hacer DELETE.

GET /api/ordenes/{id}

Obtiene una orden específica por su ObjectId de MongoDB. El ID debe ser un string hexadecimal de 24 caracteres.

Parámetros de ruta

Campo	Tipo	Requerido	Descripción
id	string (ObjectId)	✓ Sí	ID de MongoDB de la orden. Ej: 69cd838222e0b8cdf1d7998

Respuestas

Status	Descripción y ejemplo
200	Datos completos de la orden.
404	{ "error": "Orden no encontrada" } — ID no existe o malformado.

PUT /api/ordenes/{id}

Actualiza el estado o la dirección de envío de una orden existente. Se aplican validaciones de Mongoose sobre el campo estado.

Parámetros del body (todos opcionales)

Campo	Tipo	Requerido	Descripción
estado	string	○ No	Nuevo estado. Valores: pendiente, confirmada, enviada, entregada, cancelada.
direccion_envio	string	○ No	Nueva dirección de entrega.

Respuestas

Status	Descripción y ejemplo
200	Orden actualizada con los nuevos valores.
400	Estado inválido (no pertenece al enum definido en el modelo).
404	{ "error": "Orden no encontrada" }

DELETE /api/ordenes/{id}

Cancela una orden cambiando su estado a "cancelada". No elimina el documento de MongoDB, solo actualiza el campo estado.

Respuestas

Status	Descripción y ejemplo
200	{ "message": "Orden cancelada", "orden": { "estado": "cancelada", ... } }
404	{ "error": "Orden no encontrada" }

5. Microservicio Pagos

Framework: Django 5.x | Base de datos: PostgreSQL | Puerto interno: 8004
Acceso vía Gateway: <http://localhost:8000/api/pagos>

Procesa y registra los pagos del e-commerce. Genera automáticamente una referencia UUID única por cada pago. Los pagos siempre quedan en estado "completado" al crearse. Actualmente no soporta actualización ni cancelación de pagos.

Resumen de Endpoints

GET	/api/pagos	Listar pagos (filtrable por usuario)	JWT
POST	/api/pagos	Procesar nuevo pago	JWT
GET	/api/pagos/{id}	Obtener pago por ID	JWT

GET /api/pagos

Lista todos los pagos registrados. Soporta filtrado por usuario_id mediante query param.

Parámetros de query (opcionales)

Campo	Tipo	Requerido	Descripción
usuario_id	integer	○ No	Filtra los pagos de un usuario. Ej: /api/pagos?usuario_id=1

Respuestas

Status	Descripción y ejemplo
200	Lista de pagos. Incluye id, usuario_id, orden_id, monto, metodo_pago, estado, referencia y created_at.
401	Token JWT inválido.
403	Acceso directo sin X-Internal-Key.

Ejemplo de respuesta (200)

```
{
  "pagos": [
    {
      "id": 1,
      "usuario_id": 1,
      "orden_id": "69cd838222e0b8cdbf1d7998",
      "monto": 2400.00,
      "metodo_pago": "tarjeta_credito",
      "estado": "completado",
      "referencia": "cdedc6c7-90d1-48b4-9472-0aaace26ba59",
      "creado_en": "2026-04-01T23:10:17Z"
    }
  ]
}
```

POST /api/pagos

Procesa un nuevo pago. Valida todos los campos requeridos y verifica que el metodo_pago sea uno de los valores permitidos. El campo referencia (UUID) y estado ("completado") se generan automáticamente.

Parámetros del body (JSON)

Campo	Tipo	Requerido	Descripción
usuario_id	integer	✓ Sí	ID del usuario que realiza el pago.
orden_id	string	✓ Sí	ID de la orden a pagar (ObjectId de MongoDB).
monto	number	✓ Sí	Monto a pagar. Debe ser mayor a 0.
metodo_pago	string	✓ Sí	Método de pago. Valores: tarjeta_credito, tarjeta_debito, paypal, transferencia.

Ejemplo de petición

```
POST /api/pagos
Authorization: Bearer eyJ0eXAiOiJKV1Qi...
Content-Type: application/json

{
  "usuario_id": 1,
  "orden_id": "69cd838222e0b8cdbf1d7998",
  "monto": 2400.00,
  "metodo_pago": "tarjeta_credito"
}
```

Respuestas

Status	Descripción y ejemplo
201	Pago procesado. Retorna objeto completo con referencia UUID generada y estado "completado".
422	Validación fallida: campos faltantes, monto <= 0, o metodo_pago inválido.
400	JSON malformado.
401	Token JWT inválido.

Ejemplo de respuesta (201)

```
{
  "id": 1,
  "usuario_id": 1,
  "orden_id": "69cd838222e0b8cdbf1d7998",
  "monto": 2400.00,
  "metodo_pago": "tarjeta_credito",
  "estado": "completado",
  "referencia": "cdedc6c7-90d1-48b4-9472-0aaace26ba59",
  "creado_en": "2026-04-01T23:10:17Z"
}
```

Ejemplo de error (422)

```
{
  "errors": {
    "usuario_id": "Requerido",
    "monto": "Debe ser mayor a 0",
    "metodo_pago": "Debe ser uno de: tarjeta_credito, tarjeta_debito, paypal, transferencia"
  }
}
```

GET /api/pagos/{id}

Obtiene un pago específico por su ID autoincremental de PostgreSQL.

Parámetros de ruta

Campo	Tipo	Requerido	Descripción
id	integer	✓ Sí	ID numérico del pago en PostgreSQL.

Respuestas

Status	Descripción y ejemplo
200	Datos completos del pago.
404	{ "error": "Pago no encontrado" }
401	Token JWT inválido.